

AI Usage Log — Sprint 1

Project Title: LaunchpadAI — Job Search and Application Tracking Platform[cite: 1]

Course: SOEN 341 Software Process (Fall 2026)[cite: 1]

Team Member: Sudhansu Mohanty[cite: 1]

Repository Directory: AI_Log/sudhansu_mohanty/[cite: 1]

Log Entry 1: Elicitation of All AI-Generated User Stories (US-01 to US-10)

- **Task ID/Title:** Task 1.0 / Product Backlog Elicitation via Generative AI[cite: 1, 6]
- **Purpose of AI Use:** Brainstorming, initial requirements elicitation, and task breakdown for the full product backlog. The AI-generated stories (US-01 to US-10) were then refined by the team to match Supabase and Next.js technical constraints.[cite: 1].
- **Chat Link or Prompt/Response:**
 - **Tool:** Gemini / ChatGPT[cite: 1]
 - **Link:** [Insert Chat Link if available, otherwise refer to Appendix][cite: 1]
 - **Prompt:**

"Act as an Agile product manager and software requirements analyst. We are building LaunchPadAI for SOEN 341, a web-based platform designed to help job seekers manage their search activities and recruiters manage candidate applications.

Primary Users:

1. Job Seekers: Need to create profiles, upload

resumes, search jobs, submit applications, and track progress (Applied, Interview, Offered, Rejected).

2. Recruiters: Need to post jobs, manage job listings, and review incoming candidate pipelines.

Generate 10 distinct, structured user stories following the Agile template ('As a [role], I want to [action], so that [outcome]'). For each story, provide 2 to 4 concrete acceptance criteria and 2 to 3 subtasks covering database, backend API, and frontend implementation."[cite: 1]

○ **AI Response (All 10 Generated User Stories — refined versions shown):**

US-01: As a platform visitor, I want to register for an account using my email and password and select my role (`job_seeker` or `recruiter`), so that I gain access to role-tailored platform workspaces. **US-02:** As a registered user, I want to log in securely with my credentials and maintain my active session, so that I can access protected dashboards without repeatedly re-authenticating. **US-03:** As a job seeker, I want to drop and upload my PDF resume directly to my profile, so that it is securely stored in the cloud and ready to attach to job applications. **US-04:** As a job seeker, I want to manage my personal profile details, technical skills, and portfolio links, so that recruiters can review my qualifications. **US-05:** As a recruiter, I want to create and publish a new job opening with detailed specifications, so that qualified candidates can discover and apply for the role. **US-06:** As a recruiter, I want to manage my published job listings, so that I can edit posting specifications or close filled roles. **US-07:** As a job seeker, I want to search open positions by keywords and filter by location or job type, so that I can discover relevant openings. **US-08:** As a job seeker, I want to apply directly to a job opening with my stored resume attached, so that the hiring team can review my candidacy. **US-09:** As a job seeker, I want to view a

tracking dashboard of all my submitted applications, so that I can monitor my hiring progress across stages.

US-10: As a recruiter, I want to review incoming candidate applications and transition their progress stages, so that our hiring pipeline reflects current evaluations.

Each story included 2-4 acceptance criteria and 2-3 subtasks covering database, backend API, and frontend implementation. US-01 through US-03 were scoped for Sprint 1; US-04 through US-10 were deferred to Sprints 2-3.

○ Full User Story Details (AI-Generated — US-01 to US-10):

US-01 | Sudhansu Mohanty | Sprint 1 | High *Story:* As a platform visitor, I want to register for an account using my email and password and select my role (`job_seeker` or `recruiter`), so that I gain access to role-tailored platform workspaces. *Acceptance Criteria:* Signup form accepts email, password, `full_name`, and role selector toggle; enforces valid email format and minimum 8-character passwords; duplicate email registrations render inline warning; Supabase trigger auto-populates `public.profiles` with designated role. *Tasks:* Task 1.1 — SQL migration for profiles schema, `user_role` enum, and `handle_new_user` trigger; Task 1.2 — Frontend registration form UI with role toggle and error banners; Task 1.3 — Wire registration to `supabase.auth.signUp()` with metadata payload.

US-02 | Sudhansu Mohanty | Sprint 1 | High *Story:* As a registered user, I want to log in securely with my credentials and maintain my active session, so that I can access protected dashboards without repeatedly re-authenticating. *Acceptance Criteria:* Authenticate via Supabase Auth with email/password; auth state persists across navigation and refreshes via JWT session cookies; handles auth errors gracefully; redirects seekers

to `/seeker/dashboard` and recruiters to `/recruiter/dashboard`; logout purges session tokens. *Tasks:* Task 2.1 — Supabase session provider and route-guard middleware; Task 2.2 — Login UI with credential inputs, inline error validation, loading indicators; Task 2.3 — Wire login to `supabase.auth.signInWithPassword()` and implement logout handler.

US-03 | Sudhansu Mohanty | Sprint 1 | High *Story:* As a job seeker, I want to drop and upload my PDF resume directly to my profile, so that it is securely stored in the cloud and ready to attach to job applications. *Acceptance Criteria:* Interactive dropzone allows drag-and-drop or browsing for PDFs; rejects non-PDF and files exceeding 5 MB with inline feedback; stores in private Supabase Storage bucket under `resumes/{user_id}/`; updates `resume_url` on profiles record. *Tasks:* Task 3.1 — Provision private resumes bucket with RLS policies; Task 3.2 — Drag-and-drop upload UI with file size/type validation; Task 3.3 — Connect to `supabase.storage.upload()` and persist URL to profiles.

US-04 | Cheyma Abidi | Sprint 2 | Medium *Story:* As a job seeker, I want to manage my personal profile details, technical skills, and portfolio links, so that recruiters can review my qualifications. *Acceptance Criteria:* Profile page displays full name, headline, bio, and portfolio links; seeker can add/remove skill badges; RLS restricts edits to profile owner. *Tasks:* Task 4.1 — Configure RLS update policies on profiles; Task 4.2 — Profile management UI with editable fields and skill tag chips; Task 4.3 — Profile update mutation to Supabase.

US-05 | Cheyma Abidi | Sprint 2 | High *Story:* As a recruiter, I want to create and publish a new job opening with detailed specifications, so that qualified candidates can discover and apply for the role. *Acceptance Criteria:* Form contains title, company, location, job type,

description, and deadline; RLS prevents job_seeker role from creating jobs records; published postings show with status Open. *Tasks:* Task 5.1 — Design jobs table schema with recruiter FK and RLS; Task 5.2 — Job posting creation form UI; Task 5.3 — Connect form to Supabase insert.

US-06 | Cheyma Abidi | Sprint 2 | Medium Story: As a recruiter, I want to manage my published job listings, so that I can edit posting specifications or close filled roles. *Acceptance Criteria:* Dashboard lists all postings by logged-in recruiter; recruiter can toggle job status Open/Closed; closed jobs remain viewable but disable new submissions. *Tasks:* Task 6.1 — Recruiter Dashboard view with active/archived postings; Task 6.2 — Job status toggle mutation; Task 6.3 — Edit modal for descriptions, requirements, deadlines.

US-07 | Othmane Balmouddane | Sprint 2 | Medium Story: As a job seeker, I want to search open positions by keywords and filter by location or job type, so that I can discover relevant openings. *Acceptance Criteria:* Search matches keywords against titles and descriptions in real time; filters by location (Remote, Hybrid, On-site) and type (Full-time, Internship); clean empty state when no results. *Tasks:* Task 7.1 — PostgreSQL text search index and multi-filter query; Task 7.2 — Debounced search input on job board; Task 7.3 — Sidebar filter controls.

US-08 | Othmane Balmouddane | Sprint 2 | High Story: As a job seeker, I want to apply directly to a job opening with my stored resume attached, so that the hiring team can review my candidacy. *Acceptance Criteria:* "Apply Now" button prompts confirmation modal attaching default resume; creates applications record with status Applied; unique composite constraint prevents duplicate applications. *Tasks:* Task 8.1 — Applications table with unique key on (job_id, seeker_id); Task 8.2 — Application submission modal; Task 8.3 — Connect to database insert with success confirmation.

US-09 | Othmane Balmouddane | Sprint 2 | High Story:

As a job seeker, I want to view a tracking dashboard of all my submitted applications, so that I can monitor my hiring progress across stages. *Acceptance Criteria:* Dashboard displays all applications by logged-in candidate; each shows company, role, date, and stage badge (Applied, Interview, Offered, Rejected); filterable by status tab. *Tasks:* Task 9.1 — Query joining applications and jobs filtered by seeker ID; Task 9.2 — Application tracking dashboard UI; Task 9.3 — Detailed application view.

US-10 | Fatmagul Dedek | Sprint 3 | High Story: As a recruiter, I want to review incoming candidate applications and transition their progress stages, so that our hiring pipeline reflects current evaluations. *Acceptance Criteria:* View candidate list per posting with profiles and resumes; view candidate summaries and download resumes; status dropdown updates applicant stage. *Tasks:* Task 10.1 — Recruiter pipeline dashboard grouped by posting; Task 10.2 — Secure resume download via signed Supabase URLs; Task 10.3 — Candidate status update dropdown mutation.

- **AI-Suggested Content:** 10 AI-generated user stories (US-01 to US-10) covering registration, login, resume upload, profile management, job posting, job management, job search, applications, application tracking, and recruiter pipeline management. 5 additional team-original stories (US-11 to US-15) covering swipe-based job discovery, ATS resume scoring, external job API integration, deadline notifications, and job bookmarking[cite: 1, 6].
- **Validation:**
 - Cross-referenced against the SOEN 341 Sprint 1 recommended demonstration deliverables (Feature 1: User Registration/Login)[cite: 1].
 - Evaluated technical feasibility against Supabase Auth, identifying that custom user data and roles belong in a linked `public.profiles` table rather than the protected `auth.users` schema[cite: 5].
 - Reviewed during team meeting; confirmed that acceptance

criteria needed explicit validation rules (e.g., password length, unique email error handling)[cite: 6].

- **Decision: Modified before use**[cite: 1].
 - Rewrote acceptance criteria to include PostgreSQL automated trigger execution (`handle_new_user`) in Supabase to sync identity metadata into `public.profiles`[cite: 5, 6].
 - Restructured subtasks into concrete GitHub Issues: **Task 1.1** (SQL migration & role enum trigger), **Task 1.2** (Frontend registration form UI), and **Task 1.3** (Wiring Supabase Auth client with role redirects)[cite: 5, 6].
 - **Reflection:** The generated output provided a functional starting point, but it was generic[cite: 1]. Adapting it required specifying our Supabase architecture, Row Level Security (RLS) constraints, and edge-case handling to make the tasks actionable for Sprint 1 development[cite: 1, 5].
 - **Responsible Person:** Sudhansu Mohanty[cite: 1]
-

Log Entry 2: Elicitation & Definition of US-04 (Resume Upload & Storage)

- **Task ID/Title:** Task 1.0 / US-04: Resume Document Upload & Storage[cite: 1, 6]
- **Purpose of AI Use:** Requirements elicitation, storage architecture scoping, and task decomposition for resume handling[cite: 1].
- **Chat Link or Prompt/Response:**
 - **Tool:** Gemini / ChatGPT[cite: 1]
 - **Link:** [Insert Chat Link if available, otherwise refer to Appendix][cite: 1]
 - **Prompt:** (*Elicited as part of the shared 10-story prompt referenced in Log Entry 1*)[cite: 1]
 - **AI Response (Raw US-04 Output):**

User Story: As a job seeker, I want to upload my resume to my account, so that recruiters can review my qualifications when I apply for jobs[cite: 1, 6].

Acceptance Criteria:

- Candidate can select and upload a resume document from their device.
- The file name is visible on their profile.
- The uploaded file can be deleted or replaced.

Tasks:

- Configure file storage on server.
- Build file upload dropzone UI.
- Link file URL to candidate profile.

- **AI-Suggested Content:** General user story template for file uploading, visibility on profile, and basic replacement capabilities[cite: 1, 6].
 - **Validation:**
 - Verified against Sprint 1 demo requirements (Feature 2: Resume Upload / Profile Management)[cite: 1].
 - Security check: The raw output lacked access control constraints and format verification, creating vulnerabilities around unrestricted uploads and cross-user data access[cite: 5, 6].
 - Stack alignment: Evaluated Supabase Storage bucket capabilities and determined that an RLS policy scoped to `auth.uid()` was necessary to isolate resumes[cite: 5, 6].
 - **Decision: Modified before use**[cite: 1].
 - Added explicit acceptance criteria: limit file types to PDF only, enforce a 5 MB file size limit, and require storage inside a private Supabase Storage bucket[cite: 5, 6].
 - Formatted subtasks into repository work items: **Task 4.1** (Provision private bucket with authenticated RLS policy), **Task 4.2** (Client-side drag-and-drop UI with format/size validation), and **Task 4.3** (Upload API integration and metadata persistence to profile)[cite: 5, 6].
 - **Reflection:** AI suggested the overall user need, but failed to address security, storage limits, and access controls[cite: 1]. Refining the story ourselves ensured that data isolation and RLS policies were accounted for prior to implementation[cite: 1, 5].
 - **Responsible Person:** Sudhansu Mohanty[cite: 1]
-

Log Entry 3: Sprint 1 Scope Analysis and Feature Planning

- **Date:** 2026-09-26
- **Task ID/Title:** Sprint 1 Feature Scoping
- **Purpose of AI Use:** Analyzing the Sprint 1 project brief to identify required code deliverables and scope frontend vs. backend work.
- **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompt:**

"Analyse sprint 1 details and let me know what are the code implementations of all its in class-resources."
Follow-up prompts clarified scope: resume upload for Sprint 1, program management deferred; Supabase chosen for auth and database; app renamed to LaunchPadAI.

- **AI Response Summary:** Claude identified the two required Sprint 1 demo features (User Registration/Login and Resume Upload) from the project brief PDF. It recommended scoping frontend-only work first, then layering in Supabase Auth and Storage.
 - **Validation:** Cross-referenced against `class-resources/SOEN341_CareerConnect_Project_Sprint1.pdf` to confirm minimum deliverables.
 - **Decision:** Accepted AI's feature scoping. Deferred GenAI resume scoring to a future sprint. Proceeded with frontend-first approach.
 - **Reflection:** AI helped quickly parse the project brief and prioritize which features were mandatory for the Sprint 1 demo versus stretch goals.
 - **Responsible Person:** Sudhansu Mohanty
-

Log Entry 4: Frontend UI Development and

GitHub Pages Debugging

- **Date:** 2026-09-26 to 2026-09-27
- **Task ID/Title:** Frontend UI Build and Deployment Fixes
- **Purpose of AI Use:** Building the frontend UI components (login/register pages, homepage layout, testimonial panel) and debugging GitHub Pages deployment errors (HTTP 500, routing issues).

- **Prompt/Response:**

- **Tool:** Claude Code (CLI)

- **Prompts (summarized across session):**

- "A few changes to the login and sign up page, make it a horizontal rectangle and on the right 50% there are some random testimonials that rotate and fade away."
 - "Comment the see your resume through a recruiter lens, and the FAQ section."
 - "In the how it works section, the upload resume should be 3rd step instead of 2nd and make the numbers more visible and just white."
 - "There's a weird padding in the steps 1 and 2, make sure the boxes are identical in structure."
 - "Once logged in, remove the get started and sign in button on the home page."
 - "Make sure the animation is smooth when it pans down."
 - Debugging: "I'm getting code 500", "The / dashboard/resume doesn't open on the same folder so it doesn't exist", "Why was there an issue with the GitHub Pages?"

- **AI Response Summary:** Claude generated the `TestimonialPanel` component, restructured the homepage steps, fixed CSS padding inconsistencies, added conditional rendering for authenticated users, and diagnosed GitHub Pages routing issues (missing `basePath` configuration and environment variable injection in the CI workflow).

- **Validation:** Each UI change was visually verified in the local dev server. GitHub Pages deployment was tested after each fix by pushing and checking the live URL.
 - **Decision:** Accepted all UI code generated by AI. Modified the homepage step ordering manually based on team preference. AI's GitHub Pages fix (adding `basePath` and env secrets to the workflow) was adopted directly.
 - **Reflection:** AI was highly effective for rapid UI iteration and debugging build/deployment issues. The 500 error and routing problems would have taken significantly longer to diagnose manually. However, trivial follow-up prompts could have been consolidated into fewer, more specific requests.
 - **Responsible Person:** Sudhansu Mohanty
-

Log Entry 5: Supabase Authentication Integration

- **Date:** 2026-09-27
- **Task ID/Title:** Supabase Auth Client Setup and Session Management
- **Purpose of AI Use:** Configuring Supabase client credentials, implementing sign-up/login flows, session persistence, and post-login UX (dashboard page, conditional navbar).
- **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompts (summarized across session):**
 - "Now we can start integrating it to the Supabase. Right now for Sprint 1, just auth and maybe database for storing the emails and login info."
 - "Teach me how it [Supabase auth] works."
 - Provided Supabase project URL and anon key for client configuration (credentials stored in environment variables, not hardcoded).
 - "Yes turn off email confirmation, let's test it but before that let's make a page for when the user

logs in."

- "I think once we log in, it should save the session? How to do that?"
- "How does it handle the frontend when we sign in? I can still see get started or sign-in."
- "I just realized I should have an option to choose either you're a job-seeker or recruiter."
- "On the register page just a toggle option like, 'I'm a' recruiter or job-seeker."
- "How is it saved in the backend?"

○ **AI Response Summary:** Claude configured the `supabase.ts` client, implemented `signUp()` and `signInWithPassword()` calls, added session persistence using `onAuthStateChange`, created the dashboard page with logout functionality, and added a role selector toggle to the registration form. It also explained the `public.profiles` table with the `handle_new_user()` PostgreSQL trigger for storing roles.

- **Validation:** Tested registration and login locally with Supabase dashboard open to verify user records and profile rows were created correctly. Confirmed role values were persisted in `public.profiles`.
- **Decision:** Accepted AI-generated auth code. Modified the role selector styling to match the existing design language. The `handle_new_user()` trigger SQL was reviewed and applied manually in the Supabase SQL editor.
- **Reflection:** AI provided a complete end-to-end auth implementation quickly. The explanation of Supabase's `auth.users` vs `public.profiles` architecture was helpful for understanding the data model. Credentials shared in prompts should have been handled more carefully (e.g., via environment variables from the start rather than pasting inline).
- **Responsible Person:** Sudhansu Mohanty

Log Entry 6: Resume Upload Wiring to Supabase Storage

-
- **Date:** 2026-09-27
 - **Task ID/Title:** Resume Storage Integration
 - **Purpose of AI Use:** Connecting the existing drag-and-drop resume upload UI to Supabase Storage with user-scoped access policies.
 - **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompts:**
 - "How is the resume upload feature working out right now? Is it being stored somewhere profile-wise?"
 - "Wire up the resume upload to Supabase storage."
 - **AI Response Summary:** Claude identified that the existing `ResumeUploader` component was client-side only (no persistence). It then implemented `supabase.storage.from('resumes').upload()` with the file path scoped to `{user.id}/resume.pdf`, added PDF-only and 5 MB validation, and updated the profile record with the uploaded file path.
 - **Validation:** Tested upload locally; verified file appeared in Supabase Storage dashboard under the correct user folder. Confirmed RLS policy restricted access to the authenticated user's own files.
 - **Decision:** Accepted AI-generated storage integration code. Storage bucket and RLS policies were provisioned manually in Supabase dashboard following AI's instructions.
 - **Reflection:** AI correctly identified the gap between the UI component and actual persistence, and provided a working integration quickly. The RLS policy recommendation (`resumes/{auth.uid()}/*`) was critical for data isolation.
 - **Responsible Person:** Sudhansu Mohanty
-

Log Entry 7: Backlog Traceability and GitHub Issue Automation

- **Date:** 2026-09-27
 - **Task ID/Title:** Sprint 1 GitHub Issue Creation and Commit Linking
 - **Purpose of AI Use:** Automating the creation of 4 technical task issues on GitHub with commit SHA traceability, milestone assignment, and proper labeling for TA review.
 - **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompt:** Provided a detailed instruction set to inspect `git log`, verify/create labels and the Sprint 1 milestone, create 4 issues (Frontend Scaffold, Supabase Backend, Auth & Routing, Resume Storage) with commit SHAs in each body, and close them with resolution comments.
 - **AI Response Summary:** Claude ran `git log --oneline -n 15`, mapped commits to each issue (fbdc9b1, 1ca35e2, 48b50fb, a9f9e8c), created the Sprint 1 milestone and 8 labels, generated 4 issues (#19-#22) via `gh issue create`, and closed each with a verification comment.
 - **Validation:** Verified issues appeared on GitHub under the Sprint 1 milestone with correct labels, assignees, and commit references.
 - **Decision:** Accepted all generated issues. The commit-to-issue mapping was reviewed and confirmed accurate.
 - **Reflection:** AI automated what would have been a tedious manual process of creating structured issues with traceability. The prompt was highly specific, which resulted in accurate output with no modifications needed.
 - **Responsible Person:** Sudhansu Mohanty
-

Date: 2026-09-27

Prompt:

how to push to a different branch?

Date: 2026-09-27

Prompt:

is it mentioned in my ai log that i generated all the user stories 1

Date: 2026-09-27

Prompt:

yeah

Date: 2026-09-27

Prompt:

c:\Users\sudha\Downloads\userstories.md here are all the userstories

Date: 2026-09-27

Prompt:

add all the userstories i added in the .md file for transparency